MemCatalog: A Continually Evolving Memory Management System for LLM Agents

Anonymous Author(s)

Abstract

Long-term LLM agents need memory that evolves both its content and its operations. We introduce MemCatalog, a continually evolving memory management system with three levels. Raw sessions accumulate closed User–Agent conversations. First-order memory acquires source-backed user facts on demand. Second-order memory retains Agent–Memory experience: returned evidence, validated use credit, and execution outcomes/cost. First-order facts help the agent understand the user; second-order experience helps memory understand how to serve the user. For performance, single-fact retrieval, independent composition, and dependent composition partition normalized finite acyclic evidence plans and execute with experience-guided grounding. For efficiency, current evidence gaps and historical action outcomes determine a query-specific call reservation and whether further work is worthwhile. Unmet evidence obligations remain explicit when execution stops. The prototype evaluation covers eight long-term memory benchmarks. Its reported scores match or improve the strongest evaluated prior memory system on every benchmark, by an average of 2.6 points and up to 5.6 points on LME-s. On MemBench, the reported configuration uses 50.7% fewer read-path LLM calls than a Uniform-K=3 reference while attaining higher accuracy. These measurements test prototype evidence processing, workload-guided access, and selective execution; finer routing, fact-version credit, and action-value forecasts require targeted evaluation.

CCS Concepts

• **Information systems** → **Data management systems**; **Query optimization**; • **Computing methodologies** → *Natural language processing*; Multi-agent systems.

Keywords

Evolving agent memory, memory operations, second-order memory, query-driven facts, cost-aware execution

ACM Reference Format:

Anonymous Author(s). 2026. MemCatalog: A Continually Evolving Memory Management System for LLM Agents. In *Proceedings of Proceedings of the 2026 International Conference on Management of Data (SIGMOD '26).* ACM, New York, NY, USA, 13 pages. https://doi.org/XXXXXXX.XXXXXXX

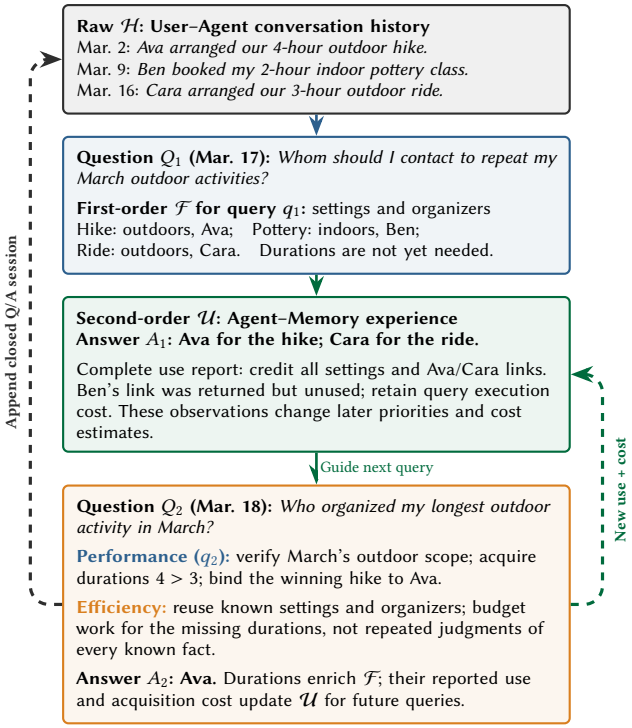


Figure 1: Two histories evolve together. The gray loop appends completed User–Agent conversations to raw sessions. Query-driven facts enrich first-order memory. The green loop retains Agent–Memory use and cost experience in second-order memory to guide later queries. Illustrative example, not an evaluation trace.

1 Introduction

Long-term LLM agents rely on memory to answer questions about a user's preferences, experiences, and changing circumstances. Memory management involves two distinct decisions: *what to store* from past interactions and *how to use it* for an arriving query. A growing fact collection helps the agent know more about the user, but does not itself determine how those facts should be accessed or

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGMOD '26, Bengaluru, India

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2026/06
https://doi.org/XXXXXXX.XXXXXXX

processed. Continually evolving memory therefore requires both retained knowledge and execution that learns from experience.

Existing systems have advanced these aspects. MemGPT manages context tiers [10], Mem0 extracts and updates facts [2], Zep maintains temporal relationships [11], and A-MEM evolves linked notes [18]. MemR^3 adapts retrieval through evidence-gap tracking and reflective control [3]. These advances motivate distinguishing content evolution and within-query adaptation from experience retained to improve later memory operations.

Two Limitations: What to Store and How to Use It. The first limitation concerns the state retained across queries; the second concerns how the current query is executed. Figure 1 illustrates both through Alice's March activities: an outdoor hike organized by Ava, an indoor pottery class organized by Ben, and an outdoor ride organized by Cara.

- **How to store: facts without the experience of using them.** Content-centered memory preserves user facts, but the fact collection alone does not record which facts were retrieved, which the agent consumed, or what providing them cost. For Alice's outdoor-contact question, memory returns three settings and three organizer links. The agent's answer names Ava and Cara, while Ben's organizer link is unused; pottery's indoor setting is still needed for exclusion. Keeping the six facts preserves what memory knows about Alice, but not what this interaction teaches memory about serving her.
- **How to use: one generic execution path for different queries.** A generic retrieve-then-answer workflow leaves the memory-side computation implicit, even when retrieval is repeated. Alice's organizer lookup needs one fact; an outdoor count needs population coverage and setting judgments; finding the organizer of the longest outdoor activity needs aligned durations and a winner-to-organizer binding. The same generic path can retrieve relevant evidence without completing these different computations. A common repetition budget also ignores the evidence state: verified durations can be reused, whereas a missing duration needs acquisition. Uniform execution can therefore hurt both performance and efficiency.

These limitations are independent: retaining experience does not specify a suitable computation, and specializing execution does not preserve what earlier queries taught the system. To address both, using memory must leave reusable experience, and later execution must act on it. This connection raises three challenges: one about what to store, and two about how to use it.

Three Challenges for Evolving Memory Management. First, represent user knowledge and operational experience separately. Raw User–Agent sessions must remain recoverable when a future query needs an unextracted detail, such as the hike's duration. Derived facts must retain their sources; Agent–Memory experience must retain query context, reported fact use, and cost. The architecture must connect these states without confusing frequently used evidence with factual truth.

Second, expose the required computation for performance. Given that state, execution must distinguish an individual fact lookup, independent evidence composition, and answer-dependent bindings. Experience can guide evidence access, but cannot replace coverage or a missing link. It must operate within a pipeline that makes these requirements explicit.

Third, allocate worthwhile work for efficiency. A suitable pipeline determines what must be resolved, not how much additional work is needed. Queries in the same plan class can have different gaps and reuse opportunities. The system must allocate effort from the current evidence and historical costs, retaining an explicit partial result when an obligation remains unresolved.

Our Proposal. We introduce MemCatalog, a continually evolving memory management system that connects these storage and execution decisions through persistent fact-use and execution experience.

For storage, a progressive hierarchy retains *raw User–Agent sessions*, *first-order facts*, and *second-order Agent–Memory experience* (Section 3). Following the query-driven acquisition principle in EUR [13], the agent's memory query for Alice's contact question acquires settings and organizers; her later comparison acquires durations from the same sessions. Closed conversations extend raw history; query execution and validated use reports extend second-order experience. First-order memory helps the agent understand the user; second-order memory helps memory understand how to serve the user.

This experience informs execution in two ways. For performance, *experience-guided memory access* (Section 4) organizes normalized plans into single-fact retrieval, independent composition, and dependent composition. The pipelines preserve source support, coverage, and bindings while remembered use guides grounding. Alice's longest-outdoor-activity organizer plan compares the 4-hour hike with the 3-hour ride, then binds the winning hike to Ava's organizer link.

For efficiency, *cost-aware execution* (Section 5) uses that plan's unresolved obligations and remembered action outcomes to reserve a query-specific budget. Known settings and organizers are reused; missing durations receive acquisition work. A later comparison can then reuse the verified durations without new semantic judgments. Execution contributes its observed outcomes, cost, and validated use credit to second-order memory, changing how subsequent queries are served. This feedback connects what memory retains with how it operates.

Contributions. The central contribution is a coupled model of conversational and operational memory, linked by query-induced user facts. The hierarchy defines what is retained; normalized plans define how evidence is processed; contextual use and cost experience guide grounding and determine how much additional work is warranted. Our prototype evaluation covers eight long-term memory benchmarks and eight LLM backbones. Its reported scores match or improve the strongest evaluated prior memory system on every benchmark, by an average of 2.6 points and up to 5.6 points on LME-s. Its measured read-path configuration uses 50.7% fewer LLM calls than a Uniform-K=3 reference on MemBench. Section 6.1 distinguishes these measurements from the generalized routing, attribution, and action-value policies.

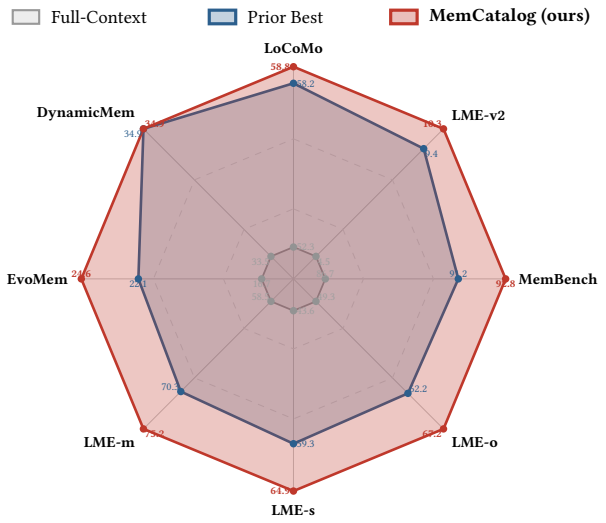


Figure 2: Per-benchmark answer scores on 8 long-term memory benchmarks. Each axis uses its own linear scale: Full-Context maps to the innermost ring, MemCatalog to the outermost regular octagon, and Prior Best falls at its position between them, so that within-benchmark differences among the three systems are directly visible. Raw scores (%) are annotated at every vertex. The prior-best system varies per benchmark (MemBench: Mem0; LME-v2: LongMemEval-Fw; the remaining six: Zep), matching Table 3.

2 Preliminaries

The user exchanges a *question* Q and an *answer* A with the agent. The agent exchanges a *memory query* q and *retrieved memory* m with the memory system:

$$\text{User} \xrightarrow{Q} \text{Agent} \xrightarrow{q} \mathcal{M} \xrightarrow{m} \text{Agent} \xrightarrow{A} \text{User}. \tag{1}$$

A question can induce several memory queries; its conversation history and each query's operational experience are recorded separately. The agent uses m to generate A; the memory support guarantee concerns m, not arbitrary claims generated in A. The query/evidence terminology is consistent with memory retrieval work [2, 3].

A memory query specifies an information need, an entity or population scope, and a temporal scope. Execution uses the source snapshot available when q arrives; newly closed sessions become available to later queries. Current and historical questions can therefore select different versions of the same fact. We model finite requests for facts or computations grounded in this history; the agent handles external knowledge and creative generation.

Retrieved memory is $m = (v, E, \sigma)$, where v is a value, fact collection, justified partial bound, or $\perp$ when no value can be established. E is a finite evidence bundle of source-backed fact versions and scope, version, and binding checks, including unresolved checks. σ is `supported`, `partial`, or `unknown`. A count needs evidence that covers its requested population; a dependent result needs evidence for each required link. Supported means all required checks are discharged; partial means some evidence is established but an obligation remains; unknown means no source support for the result is established. A verified empty scope is supported absence, not unknown. An unresolved check is not evidence that its predicate is false. These states refer to a validated plan and source snapshot, not a promise of perfect natural-language interpretation.

Experience is retained per user: later queries can benefit from earlier fact use and execution outcomes without mixing different users' states. Table 1 lists the core symbols; each operational quantity is defined where it is used.

Table 1: Core notation for the two interfaces and the evolving memory hierarchy.

Symbol	Meaning
Q, A	User question Q; agent answer A
q, m	Memory query q; retrieved memory m
$\mathcal{H}, \mathcal{F}, \mathcal{U}$	User–Agent sessions $\mathcal{H}$; query-induced facts $\mathcal{F}$; Agent–Memory experience $\mathcal{U}$
$\mathcal{M}$	User's memory state $\mathcal{M} = (\mathcal{H}, \mathcal{F}, \mathcal{U})$
f	A first-order fact f, with source and time
P	Query plan $P = (R, D, g)$: requests R, dependencies D, result operation g
E^r, E^u	Returned fact-version IDs E^r; validated reported-use IDs $E^u \subseteq E^r$
ℓ, C	Unresolved-check fraction ℓ; memory-query LLM-call cost $C(q)$
$K(q)$	Current additional-call reservation $K(q)$ for query q
λ	Diagnostic–cost tradeoff weight λ

3 Architecture

MemCatalog retains User–Agent conversation history in raw sessions and Agent–Memory experience in second-order memory. First-order facts connect them: queries extract facts from raw sessions, and fact use produces operational experience. User knowledge remains distinct from experience about serving the user.

3.1 From Sessions to Facts to Use Experience

For each user, the memory state is

$$\mathcal{M} = (\mathcal{H}, \mathcal{F}, \mathcal{U}). \tag{2}$$

The progression $\mathcal{H} \to \mathcal{F} \to \mathcal{U}$ links conversations, derived facts, and experience of using those facts; it is not an execution order. $\mathcal{U}$ also retains execution outcomes and cost when no fact is returned. All three levels are isolated by user.

Raw sessions: User–Agent history. $\mathcal{H} = (s_1, \ldots, s_n)$ is an append-only sequence of closed, timestamped conversations, including questions Q, answers A, and speaker identities. Sessions remain available, including details that no previous query needed. Agent text retains its speaker identity: suggestions and generated claims are not independent user evidence. They require user confirmation or lineage to already grounded assertions; storing an answer does not create new source support.

First-order memory: what the agent knows about the user. $\mathcal{F}$ contains source-backed assertion records f, normally with a

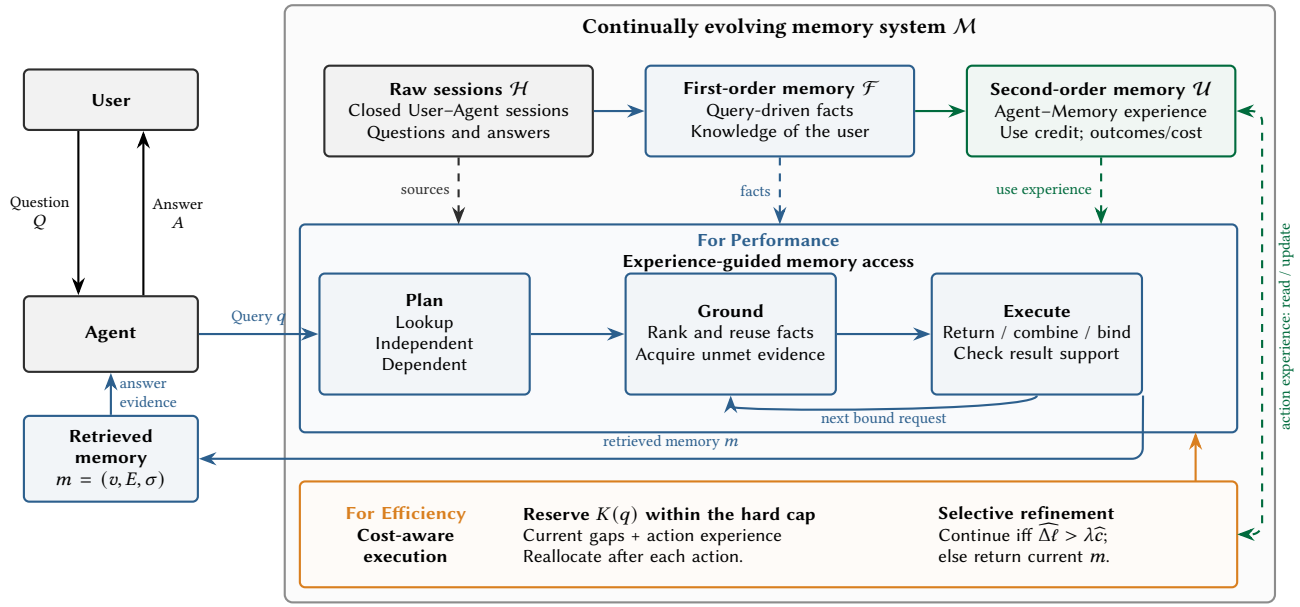


Figure 3: State, performance, and efficiency occupy separate bands. Raw sessions retain closed User–Agent conversations; first-order memory retains query-driven facts; second-order memory retains Agent–Memory experience. The performance pipeline plans, grounds, and executes, preserving source support, coverage, and bindings. Cost-aware execution controls its work through query-specific reservations and selective refinement. The local blue loop resolves the next bound request. Dashed paths show state access and feedback; the shared green outer lane reads and updates action outcomes/cost.

(subject, predicate, object) payload. Each record has a stable fact-version ID, source spans, and temporal qualifiers; equal triples with different qualifiers remain distinct. Event identifiers connect facts about the same activity across sessions. Extraction is query-driven: an organizer query can acquire an event's organizer, while a later comparison acquires its duration from the same source. Facts accumulate as the workload reveals what is needed.

Second-order memory: Agent–Memory experience. $\mathcal{U}$ retains memory-query context, returned evidence, support outcomes, action costs, and validated credit for agent-reported fact use. It distinguishes exposure from reported consumption and stores both execution records and statistics derived from them. Repeated organizer requests teach the system which event links to prioritize and reuse. The corresponding costs teach it where another execution is useful.

Example 1: from a conversation to operational experience

1. Raw conversations. Alice says on March 2, *Ava arranged our 4-hour outdoor hike at Pine Trail.* On March 9, *Ben booked my 2-hour indoor pottery class.* On March 16, *Cara arranged our 3-hour outdoor ride at Lake Park.*

Example 1 (continued)

2. The question determines extraction. On March 17, Alice asks, *Whom should I contact to repeat my March outdoor activities?* The agent requests March activities, settings, and organizers:

Activity ID	Setting	Organizer
`hike-1`	outdoors	Ava
`pottery-1`	indoors	Ben
`ride-1`	outdoors	Cara

These six entries, together with a verified three-activity scope, support the response. The facts include `(hike-1, organizer, Ava)`. Unneeded durations remain in raw sessions.

3. Retrieval differs from use. The agent answers, *Ava for the hike; Cara for the ride.* A complete use report credits all three settings, including pottery's exclusion, and the Ava/Cara links. Ben's organizer was returned but unused. Second-order memory records these IDs, context, and cost. Unused facts remain available.

4. All three levels evolve. The March 18 question about the longest outdoor activity induces a memory query that acquires durations, compares $4 > 3$, and resolves its organizer, Ava. Durations enrich $\mathcal{F}$; their use and acquisition cost enrich $\mathcal{U}$. The completed question/answer session is appended to $\mathcal{H}$ when it closes.

3.2 How Content and Operations Evolve

When q arrives, its source snapshot is fixed and required evidence is checked against $\mathcal{F}$. Missing properties, links, versions, or population coverage identify which source regions must be examined. Newly acquired facts enrich $\mathcal{F}$ after verification. Acquisition reconciles identities, normalizes dates and units, and retains temporal

versions. Relative dates use the source session's timestamp. Multiple valid objects can coexist; an explicit update to a single-valued property creates a new version rather than erasing its history.

After execution, $\mathcal{U}$ records q's context, returned fact-version IDs E^r, support diagnostics, and costs. E^r includes the facts exposed in m's evidence bundle, not every fact inspected internally. A complete agent use report identifies q and its consumed fact or derived-result IDs. For a consumed result, memory expands its derivation to the supporting facts, including negative evidence that excludes a candidate from a count. Every such supporting fact is included in E^r. Membership, lineage, and source/version checks against q's snapshot validate the resulting $E^u \subseteq E^r$; each fact receives at most one use credit per query, with idempotent updates by query and fact-version ID. Returning a result alone does not imply use, and credit does not establish a causal contribution to answer correctness. Incomplete or missing reports leave use unobserved; execution records are retained independently. A closed User–Agent session is appended to $\mathcal{H}$. Later queries revalidate cached scope and version checks affected by the new session; old assertions and their provenance are not erased.

Figure 3 shows how this experience affects the next query. For performance, it guides evidence selection and reuse within a dependency-specific pipeline (Section 4). For efficiency, it guides the amount and location of additional execution (Section 5). The operations evolve through their priorities, reuse decisions, and budgets; source evidence continues to determine what a supported result can assert.

4 For Performance: Experience-Guided Memory Access

A memory query needs evidence organized into its required computation, not just relevant passages. The pipelines distinguish a fact lookup from independent and dependent composition. Second-order experience guides grounding without changing support requirements.

4.1 Fact Lookup and Evidence Composition

The three plans are *single-fact retrieval* (Lookup), *independent composition* (Independent), and *dependent composition* (Dependent). The first is the base case. The other two distinguish whether one evidence request needs a value produced by another. Independent composition includes counts, comparisons, summaries, and tuples of unrelated facts; it is not restricted to numeric aggregation. Dependent composition includes answer-dependent entity or time bindings, with independent branches allowed inside the plan.

The compiler produces $P = (R, D, g)$ for q. $R = (r_1, \ldots, r_d)$, $d \geq 1$, contains bounded lookup or population-block request templates. g is a type-checked result operation, and $(j, i, p) \in D$ binds the output of r_j to input slot p of r_i. Each template carries q's source snapshot and declared scopes. Slots accept scalar or finite-set values and have at most one binding. Normalization exposes cross-goal bindings and removes unused requests and identity-only wrappers. Candidate enumeration, independent evaluation, and reduction form one population block; a result that determines a later request creates an explicit edge. The graph is finite and acyclic.

A plan is *atomic* when it requests one individual fact and g projects its value. Population operations remain non-atomic even with zero or one observed candidate. Thus

$$\rho(P) = \begin{cases} \textsc{Lookup}, & D = \emptyset,\ P \text{ atomic}, \\ \textsc{Independent}, & D = \emptyset,\ P \text{ non-atomic}, \\ \textsc{Dependent}, & D \neq \emptyset. \end{cases} \tag{3}$$

Proposition 1 (Complete and disjoint plan classes). *Every well-formed normalized finite acyclic evidence plan belongs to exactly one class in Eq.* (3).

Proof. A plan either has a dependency edge or has none. In the latter case it is either atomic or non-atomic. The cases are exhaustive and mutually exclusive. □

This is a guarantee about plans, not intrinsic labels for natural-language questions. For the longest-outdoor-activity organizer, we use a goal-directed plan: select the event, then bind it to an organizer lookup. An alternative that fetches every organizer before selecting could be independent. Both plans represent the same question, but incur different work. The compiler exposes the chosen computation and validates entity/time scopes, bindings, and g. Model-assisted compilation is charged when needed; ambiguous scope is clarified or reported as a gap. The partition does not prove that compilation always interprets a question correctly.

4.2 Learning to Ground and Execute

For a fact version f and query context z, $\mathcal{U}$ retains a feedback-observed exposure count $n^r(f, z)$ and a credited-use count $n^u(f, z)$. Contexts identify topic, temporal intent, and evidence requirements. A complete agent use report increments n^r once for each returned fact and n^u once for each fact in the validated reported-use set. Thus $0 \leq n^u \leq n^r$. Incomplete or missing reports update neither count: unknown use is not treated as non-use. All retrieval traces and execution costs are still retained.

With positive use/non-use prior counts α_0, β_0, contextual use propensity and grounding priority are

$$\begin{aligned} u(f \mid q) &= \frac{\alpha_0 + n^u(f, z(q))}{\alpha_0 + \beta_0 + n^r(f, z(q))}, \\ s(f \mid q) &= \mathrm{rel}(f, q)\, u(f \mid q). \end{aligned} \tag{4}$$

Here $\mathrm{rel}(f, q) \in [0, 1]$ is current query relevance. The ratio estimates reported use conditional on complete feedback, not factual truth or the probability of a correct answer. Source and version checks establish support independently. When a grounding batch has eligible alternatives, one position is reserved for a relevant, least-exposed fact in the current context, breaking ties by fact-version ID; the other positions follow s. The reserved position admits underexposed eligible evidence. Priorities affect the order of work, never the population or dependency obligations of the plan.

In Example 1, with illustrative priors $\alpha_0 = \beta_0 = 1$, the Ava/Cara organizer links have estimated use propensity $2/3$ and Ben's has $1/3$ in a matched contact context. This changes the priority of equally relevant facts, not their truth; an indoor-activity request can still favor Ben.

All pipelines use $E_i = \mathrm{Ground}(r_i, \mathcal{M})$ against q's fixed source snapshot. Grounding checks identity, scope, source, and version,

reuses eligible facts, and acquires unmet evidence from raw sessions (Section 3.2). Experience orders facts and source regions without replacing those checks.

Single-fact retrieval: locate, verify, return. For an individual target, select its source-supported temporal value $v = \text{Select}(r, E)$. Missing facts are not automatically false; unresolved identity, conflicting versions, or absent support yields a gap. Verified cached evidence can be returned without another semantic judgment.

Independent composition: ground, evaluate, combine. With no cross-request bindings, ground the requests independently and apply g to their values. A tuple can combine a preference and an allergy. For a population block, establish its scope, deduplicate identities, then obtain each candidate's evidence. For the current population request r,

$$\begin{aligned} x_e &= \text{Eval}(r, e, E_e), \\ v &= \text{Combine}(r, \{(e, x_e) : e \in \text{scope}(r)\}). \end{aligned} \tag{5}$$

x_e is a verified value or predicate outcome, or $\perp$ if unresolved. Counts use yes/no judgments; comparisons align normalized values with their candidates; summaries retain source support. Combine implements the request's count, comparison, tuple, or source-grounded synthesis; g combines request outputs. Independent judgments may execute in parallel. A closed empty scope has a defined result, such as count 0 or no matching event, rather than an unknown value. Comparisons retain all tied winners; a later lookup instantiates one request per bound winner rather than silently choosing one.

An exact population result needs complete coverage and every required judgment. With h known positives and u unresolved judgments in a closed scope, return $[h, h+u]$ with partial support. Without coverage, only h is a justified lower bound. A highly ranked subset cannot establish an exact count.

Dependent composition: resolve, bind, continue. A node is ready only when every required predecessor has $\sigma_j = \texttt{supported}$. Evaluate ready nodes in topological order, then finalize g only when its required inputs are supported:

$$\begin{aligned} r_i^* &= r_i[p \leftarrow v_j : (j, i, p) \in D], \\ m_i &= (v_i, E_i, \sigma_i) = \text{Solve}(r_i^*, \mathcal{M}), \\ v &= g(v_1, \ldots, v_d). \end{aligned} \tag{6}$$

Solve executes the normalized retrieval or independent block. Ready branches can run concurrently; a bound request returns to grounding for its newly determined target. A blocked node is not passed to Solve: it retains the unresolved predecessor obligation. Available supports yield a partial response; no established evidence yields unknown. A definitive final value is computed only from the inputs required by g. A supported predecessor with no matching target instead propagates supported absence and skips the target lookup, as specified by g; it is not an unresolved binding.

Support invariant. The final $m = (v, E, \sigma)$ is supported only if its plan is validated and its required identity, source, version, coverage, value, binding, and result checks are discharged; E carries their supports, including the dependency ancestors of projected outputs and their selection and exclusion evidence. Otherwise the result remains partial or unknown. Conditional on correct primitive checks and scope interpretation, this rule is preserved by independent reduction and, inductively, by dependency-ordered binding. Neither a high use score nor a larger budget can certify an unresolved obligation. This is a support guarantee relative to the validated plan, not a guarantee of error-free LLM extraction.

Example 2: one fact, independent evidence, dependent evidence

Use Alice's three March activities from Example 1.

Lookup. *Who organized the hike?* Verify `hike-1`'s organizer and return Ava.

Independent. *How many activities were outdoors?* Evaluate hike → yes, pottery → no, ride → yes: 1 + 0 + 1 = 2. If pottery's setting were unknown, return $[2, 3]$.

Dependent. *Who organized the longest outdoor activity?* Acquire durations: hike, 4 hours; ride, 3 hours. Select `hike-1`, bind it to the organizer lookup, and return Ava. An unresolved winner cannot justify choosing Ava over Cara.

Retain new durations in $\mathcal{F}$. A consumed-result use report credits its supporting facts, including exclusion evidence.

5 For Efficiency: Cost-Aware Execution

Plan class determines the required computation, not its expense. One independent-composition query can reuse every value; another must acquire a missing property or revisit an ambiguous source. Cost-aware execution allocates work from the current evidence state and second-order action experience, then revises that allocation as evidence changes.

5.1 Query-Specific Budget Allocation

An action is a bounded unit of evidence work: acquire a source-backed property, verify a disputed value, resolve a plan block, or check its result. Let $\tau(q)$ be the executed action sequence, with $\tau_j(q)$ its jth occurrence. In memory-side LLM calls,

$$C(q) = C_{\text{plan}}(q) + \sum_{j=1}^{|\tau(q)|} c(\tau_j(q)). \tag{7}$$

Each invocation is charged exactly once, including repeated actions. Semantic synthesis or validation by g is an execution action, not free finalization. Acquisition and verification are not free write-path overheads. Deterministic reads, selection, and arithmetic use zero LLM calls but can still incur latency. Agent-side query formulation and final answer generation are accounted for separately in an end-to-end ledger.

The executor maintains evidence obligations for scope coverage, values, versions, bindings, and result checks. For the current snapshot of instantiated obligations Ω, let $\ell(\Omega)$ be the unresolved fraction, with $\ell(\varnothing) = 0$; conflicts leave a check unresolved. It is an observable evidence-gap diagnostic, not online accuracy or a calibrated error probability. Neither agreement nor an empty diagnostic after failed plan compilation certifies a result. Ambiguous compilation remains an unresolved check.

An action's observed reduction compares the same pre-action set: $\Delta\ell(a) = \ell_{\text{before}}(\Omega) - \ell_{\text{after}}(\Omega)$. Newly discovered candidates add obligations to the next decision's snapshot, not to this comparison's denominator. Thus useful scope discovery is not penalized

merely for exposing additional work. The executor then checks the expanded set before certifying a result or allocating the next action.

For each executed action, $\mathcal{U}$ stores its context, obligation snapshot, observed reduction, and actual call cost. A context signature contains action kind, plan class, unresolved-check types, candidate count, dependency depth, and cache availability (all, some, or none). To estimate an available action's diagnostic reduction $\widehat{\Delta\ell}(a)$ and cost $\widehat{c}(a)$, the executor averages matched historical reductions and costs with one declared prior observation. Detailed signatures match these fields exactly. It backs off from the detailed signature to the same user's action-kind/plan-class history, then to the prior. Matching and backoff use these declared fields and order. These are predictions from executed actions, not counterfactual labels.

The cold prior supplies the template's declared call count and half the fraction of unresolved checks it could discharge with available sources, or zero when none is available. This is an initial assumption, not a measured success rate; its weight decreases with matched experience. Each action declares an enforced maximum reservation $c_{\max}(a)$: $0 \le c(a) \le c_{\max}(a)$, distinct from its predicted mean cost. An action that reaches its reservation returns its current evidence state.

From dependency-ready actions, the executor constructs a worklist $\mathcal{A}_+(q)$ in descending predicted net benefit $\widehat{\Delta\ell}(a) - \lambda\widehat{c}(a)$. Only positive-benefit actions that fit the remaining hard cap are admitted. The current additional-call budget is

$$K(q) = \sum_{a \in \mathcal{A}_+(q)} c_{\max}(a) \;\le\; K_{\max}(q) - C_{\text{spent}}(q). \tag{8}$$

Here finite $K_{\max}(q) \ge 0$ caps all memory-side calls, including planning, and C_{spent} records calls already paid. Planning is invoked only under a reservation that fits the cap; otherwise the system reports the unresolved plan rather than overspending. $\lambda \ge 0$ prices diagnostic reduction against calls. The worklist, $C_{\text{spent}}(q)$, and $K(q)$ refer to the current decision, not fixed query-wide constants. After each action, unused reservations are released and the worklist is rebuilt from the new evidence state.

5.2 Selective Refinement and Experience Feedback

When admissible actions exist, the next action maximizes predicted net benefit:

$$\begin{gathered} a^* \in \underset{a \text{ admissible}}{\arg\max} \{\widehat{\Delta\ell}(a) - \lambda\widehat{c}(a)\}, \\ \text{continue} \iff \widehat{\Delta\ell}(a^*) > \lambda\widehat{c}(a^*). \end{gathered} \tag{9}$$

If no action is admissible, execution stops. Admissibility requires available inputs and a reservation that fits the remaining cap. A prerequisite sequence with delayed benefit can be evaluated as one bounded bundle, charging every invocation within it. A bundle's first action must be ready, with later inputs supplied by earlier actions in the bundle. The rule is a local, experience-based decision, not a claim of globally optimal scheduling or guaranteed accuracy improvement. Free deterministic work is not repeated on unchanged inputs. Repeated semantic judgments consume calls from the hard cap. With a finite plan and source snapshot, these rules prevent an unresolved gap from inducing an unlimited retry loop.

Execution reuses verified subresults and spends only on unresolved work. Semantic disagreement can trigger another judgment; shared unsupported evidence calls for source acquisition instead of voting. If an input changes, only its dependent descendants are invalidated and recomputed. Exhausting the cap or declining further work does not convert a gap into a supported result: σ remains partial or unknown as appropriate.

Example 3: reuse completed work, resolve the new gap

Warm query. Alice again compares the outdoor activities. Reuse verified settings and durations: $4 > 3$, with $K(q) = 0$ additional memory-side calls. Agent answer generation is separate.

New gap. A closed March 23 session says, *We planned an outdoor picnic, but rain changed our plans.* Its follow-up says, *We ate inside River Cafe instead.* The two descriptions refer to one outing, not two activities. Alice's March 24 count needs its actual occurrence and setting, not merely the planned picnic.

Selective action. Verify the outing and acquire its actual setting when predicted benefit exceeds cost, charging extraction/verification calls. Reuse the three earlier judgments; update only the new outing: $1 + 0 + 1 + 0 = 2$.

Unavailable evidence. Without the follow-up, the closed four-candidate scope supports $[2, 3]$, not an exact count. Repeating the planned-picnic fragment cannot resolve this gap.

Query totals $C(q)$, action outcomes, and diagnostic changes update $\mathcal{U}$ independently of whether the agent supplies a use report. Validated reported use updates the fact-priority statistics in Section 4.2. The records distinguish cold acquisition from warm reuse and guide the next query's priorities and budget. Acquired facts evolve content; remembered use, outcomes, and cost evolve memory operations.

6 Experiments

We examine evidence organization and pipelines (Exp-1–3 and Exp-5), persistent access experience (Exp-5–6), and selective execution (Exp-4, Exp-7, and Exp-8). Component controls complement overall scores; Section 6.1 defines which mechanisms the prototype measures.

6.1 Experimental Setup

Evaluation Scope. The evaluated prototype uses POINT, AGGREGATE, and COMPARE as benchmark patterns, not the normalized dependency classes in Section 4. It acquires session-level facts on first access, uses coarser workload experience than the fact-version credit specified here, and applies pattern-dependent reservations with agreement-based stopping. These measurements test evidence organization, workload-guided access, and selective execution, not the formal model's finer acquisition, normalized-plan routing, fact-version credit, or action-value estimator. Those policies require targeted evaluation. In the tables, a benchmark query is one question Q; it is not an internal memory query q. Uniform-K=3 names three read-path executions per benchmark question, not the adaptive reservation $K(q)$.

Table 2: The eight benchmarks, grouped by the memory ability they stress. Their history lengths cover a broad range. Cross-benchmark differences alone do not isolate representation effects from context length or task difficulty.

Benchmark	Memory ability	History
LoCoMo [8]	Multi-session dialogue	~9K
LME-v2 [16]	Entity-centric recall	~115K
MemBench [12]	Typed query patterns	~100K
LME-o [16]	Long-horizon retention	~40K
LME-s [16]	Long-horizon retention	~115K
LME-m [16]	Long-horizon retention	~1.5M
EvoMem [15]	Evolving user state	~50K
DynamicMem [17]	Evolving user state	~127K

Datasets. Table 2 groups eight benchmarks by the ability they stress. LoCoMo [8] and LME-v2 [16] test multi-session recall. LME-o, LME-s, and LME-m [16] evaluate related tasks at increasing history lengths. EvoMem [15] and DynamicMem [17] contain repeated questions over evolving user state. MemBench [12] contains 300 benchmark questions in the reported evaluation and supplies the pattern groups for the routing and cost controls. These settings expose different pressures: source access over long histories, computation over multiple facts, and accumulated experience across a user's requests.

Baselines. The seven baselines are Full-Context [7], RAG [6], MemGPT [10], Mem0 and Mem0^g [2], Zep [11], and LongMemEval-Fw [16]. They provide, respectively, full-history prompting, passage retrieval, context tiers, consolidated facts and entity relations, temporal graphs, and an indexing–retrieval–reading pipeline. The reported comparison uses released implementations under a shared backbone. Prompts and implementation choices also differ, so component ablations complement the overall system comparison.

Metrics and Settings. Multiple-choice tasks use exact match; open-answer tasks use F1, with phrase-level normalization for LME-v2 and token-level normalization elsewhere. Comparisons are within a benchmark's metric scale. The default backbone is GPT-5.5; Exp-2 covers four closed-source and four open-source models [1, 4, 9]. The prototype uses Python and an OpenAI-compatible client. Reported control settings are reused across benchmarks and backbones. Read-path calls are counted per benchmark question and exclude fact acquisition; Exp-4 distinguishes this metric from the full memory-query ledger in Eq. (7). The reported totals are not an acquisition-inclusive ledger or a measurement of separately accounted Agent-side calls.

6.2 Exp-1. Answer Scores Across Memory Abilities

Table 3 shows that MemCatalog matches or improves the strongest evaluated baseline in every column. The mean benchmark-score margin is 2.6 points; this is an arithmetic mean of within-benchmark differences, not a pooled accuracy across different metrics. The distribution of gains is more informative than the mean alone.

Evidence organization matters most over long histories. The margins on LME-o, LME-s, and LME-m are 5.0, 5.6, and 4.9 points. A query over many sessions has to recover the right entities and temporal versions before computing its answer. The architecture in Section 3 separates this source history from reusable fact values; the pipelines in Section 4 then expose coverage and binding obligations. This reduces the amount of evidence alignment left implicit in a long generation, a difficulty also observed in long-context evaluations [5, 7]. The maintained margin at the largest scale is consistent with this separation, rather than a benefit restricted to short histories. Exp-5 supplies the complementary evidence that removing the fact representation or explicit operators lowers accuracy.

Overall scores do not identify which computations improve. The margins are 0.6 points on LoCoMo and 1.6 on MemBench. These averages do not establish the workloads' plan-class distributions. An individual lookup requires identity/version selection and source support, whereas a population request also needs candidate computation. On LME-v2, 10.3 versus 9.4 gives a 9.6% relative improvement, but its phrase-level F1 scale prevents interpreting the absolute gap as equivalent to the multiple-choice gaps. Exp-3 tests the within-pattern distinction more directly.

Aggregate state accuracy can hide operational evolution. On EvoMem, MemCatalog gains 2.5 points over the strongest baseline; on DynamicMem, it ties that baseline at 34.9. This tie is important: repeated interaction does not imply a universal aggregate gain. An overall score mixes early queries with sparse experience and later queries that can reuse it. The position analysis in Exp-6 examines that trajectory, while Exp-5 tests whether workload experience matters at unchanged call totals.

Finding 1: The overall advantage is strongest on long-horizon workloads. Representation and pipeline ablations provide a more direct link to the technical mechanisms than the aggregate ranking alone.

6.3 Exp-2. Transfer Across Backbones

We hold MemBench and the reported configuration fixed and compare against the strongest baseline rerun with each backbone.

The gains in Table 4 range from 1.6 to 5.9 points for closed-source models and from 5.8 to 7.9 for open-source models. Their group means are 3.9 and 7.0 points. Within each group, the gain increases as the prior-baseline score falls. Thus the advantage is not restricted to the default model's particular way of using memory.

Explicit evidence processing provides a possible explanation: memory identifies values and exposes computation instead of leaving entity alignment, counting, and answering to one generation. This can help a weaker backbone, but source ambiguity and semantic judgments remain. The gains support transfer across models; they do not identify which internal capability causes the difference.

Finding 2: Gains transfer across all evaluated backbones and are larger for lower-scoring models within each group, consistent with explicit evidence processing reducing the burden on the backbone.

Table 3: Reported prototype answer scores across eight long-term memory benchmarks. MemCatalog matches or improves the strongest evaluated baseline in each column; the largest margins are on the long-horizon variants.

Method	Conversational memory			Long-horizon memory			Personalized state	
	LoCoMo	LME-v2	MemBench	LME-o	LME-s	LME-m	EvoMem	DynamicMem
Prompting baselines								
Full-Context [7]	52.3	4.5	86.7	49.3	43.6	58.5	16.7	33.5
RAG [6]	53.1	4.9	87.8	50.8	45.1	59.1	17.3	34.1
Memory systems								
MemGPT [10]	54.5	6.3	89.7	54.1	48.4	61.1	18.0	34.3
Mem0 [2]	56.4	7.2	91.2	58.3	54.2	65.2	20.4	34.5
Mem0^g [2]	57.1	7.5	90.8	60.1	56.7	67.1	21.2	34.7
Zep [11]	58.2	8.1	90.4	62.2	59.3	70.3	22.1	34.9
LongMemEval-Fw [16]	57.3	9.4	90.1	61.7	58.8	69.4	21.6	34.6
Ours								
MemCatalog	**58.8**	**10.3**	**92.8**	**67.2**	**64.9**	**75.2**	**24.6**	**34.9**

Table 4: Accuracy on MemBench across backbones. Best prior is the strongest of the seven baselines of Table 3 re-run on that same backbone. The reported gain is positive for every evaluated backbone and larger for lower-scoring backbones within each group.

Backbone	Best prior	MemCatalog
Closed-source		
GPT-5.5	91.2	**92.8**
Claude-3.5-Sonnet	87.4	**91.4**
Gemini-1.5-Pro	85.6	**89.7**
GPT-4o	82.8	**88.7**
Open-source		
DeepSeek-V4	82.7	**88.5**
Llama-3.1-70B	79.8	**86.5**
Qwen3-14B	77.4	**85.1**
Qwen3-8B	73.4	**81.3**

Table 5: Reported per-pattern scores on MemBench. Pattern labels differ from normalized plan classes; grouping/averaging remains unreconciled with Table 3.

Method	POINT	AGGREGATE	COMPARE
Prompting baselines			
Full-Context [7]	90.9	64.4	84.2
RAG [6]	91.8	67.1	85.8
Memory systems			
MemGPT [10]	92.9	68.5	87.0
Mem0 [2]	93.9	70.2	88.1
Mem0^g [2]	94.1	71.9	85.4
Zep [11]	94.3	71.2	85.4
LongMemEval-Fw [16]	93.8	70.8	84.7
Ours			
MemCatalog	**94.6**	**78.0**	**91.2**

6.4 Exp-3. Accuracy by Evidence Computation

Within Table 5, MemCatalog exceeds the strongest prior score by 0.3 points on point lookup, 6.1 on aggregation, and 3.1 on comparison. The difference between these gaps links the results to the pipelines more clearly than a single overall score.

Aggregation needs coverage, not just relevance. A count can be wrong even when the retrieved passages are topically relevant: candidates can be omitted or duplicated, and a mentioned event may not satisfy the requested predicate. The population pipeline in Eq. (5) separates enumeration, per-candidate judgment, and tallying. In Alice's example, pottery must be examined and excluded, not simply dropped because the desired answer concerns outdoor activities. Once each judgment is supported, arithmetic need not be another unstructured model inference. The larger aggregation gap is consistent with making these operations explicit, although the table does not individually measure coverage errors and predicate errors.

Comparison needs aligned values; composition needs bindings. A comparison must associate each normalized value with its event before ordering them. Retrieving both durations without that alignment can still select the wrong activity. For independent candidates this is an independent reduction; when its winner determines an organizer request, Eq. (6) adds a dependent binding. The comparison scores support explicit value processing, but do not separately test that binding step. Establishing the new plan boundary requires dependency annotations, not relabeling the original comparison group as DEPENDENT.

A separate benchmark trace in Table 6 shows the distinction: Full-Context leaves counting implicit and returns 5; Zep leaves the predicate unresolved and returns 4; the prototype makes candidate decisions explicit and returns 3 after two agreeing executions. The trace illustrates why a relevant representation alone is not a complete computation. It also separates the roles of access experience, candidate evaluation, and stopping, without proving that each one independently caused the correct answer.

Table 6: Reported aggregation trace: implicit counting and unresolved predicates contrast with explicit candidate decisions and tallying. The stopping behavior is from the measured prototype.

System	Retrieval and reasoning	Calls	Answer
Full-Context	count left implicit	1	5
Zep [11]	predicate unresolved	1	4
MemCatalog, measured execution steps			
Planner routes on the quantifier marker			
Workload experience prioritizes resolved entities			
Operator decides each candidate, then tallies			
Executor withholds the third call on agreement			
MemCatalog	explicit per-item decision	2	**3**

Table 7: Reported read-path calls on MemBench, excluding fact acquisition. Agreement stopping saves calls with unchanged aggregate accuracy in this comparison; it does not establish that budget choices are generally accuracy-neutral.

Configuration	Calls per question			Total	Acc.
	POINT	AGGREGATE	COMPARE		
Uniform, one call	1.00	1.00	1.00	300	86.7
Uniform, three calls	3.00	3.00	3.00	900	89.1
MemCatalog, budget only	1.00	3.00	3.00	532	92.8
MemCatalog	1.00	2.30	2.18	**444**	**92.8**

The reported per-pattern scores have an unresolved grouping/averaging provenance: they cannot be read as a micro-accuracy decomposition of Table 3 under shared class weights. The comparisons above are within Table 5; attributing a fraction of the overall gain to a pattern requires the original per-query records.

Finding 3: The largest within-pattern gap is on aggregation, where coverage and candidate decisions must precede tallying. Comparison supports explicit value alignment, not a separate test of dependent bindings.

6.5 Exp-4. Where Execution Savings Come From

Table 7 separates two sources of read-path savings: selecting which queries receive repeated execution, and stopping before all allocated repetitions are spent.

Selective allocation avoids paying for uniform repetition. Uniform-K=3 uses 900 calls at 89.1 accuracy. The budget-only prototype uses 532 calls at 92.8: 368 fewer calls (40.9%) with a higher score. This comparison changes evidence processing as well as allocation, so it does not attribute the accuracy gain to budget selection alone. It does show that a larger global call count is not sufficient to recover the prototype's accuracy. Repetition must be attached to useful evidence work.

Stopping removes work after the allocation decision. Agreement stopping reduces 532 calls to 444, saving another 88 (16.5% of the budget-only count), with the same reported aggregate accuracy. Point lookups remain at one call; aggregation and comparison average 2.30 and 2.18 instead of three. These unequal realized costs show that an allocated maximum need not be fully spent. In total, 456 calls are saved relative to Uniform-K=3, giving the reported 50.7% reduction. The equality of aggregate scores does not imply identical predictions.

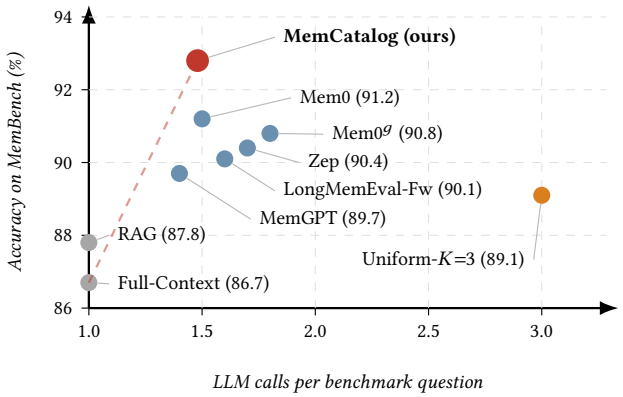


Figure 4: Reported read-path cost versus accuracy on MemBench (GPT-5.5, n=300), excluding fact acquisition. MemCatalog has higher accuracy than the evaluated prior systems and lower read-path cost than Uniform-K=3. These points do not measure the acquisition-inclusive action policy of Section 5.

These controls motivate the action-level decision in Section 5: allocate to unresolved work, not just a query label, and compare another action's predicted evidence-gap reduction with its cost. Alice's verified duration comparison needs no new semantic judgment; her ambiguous outing requires new source evidence rather than more votes over the same fragment. Agreement-based savings support selective stopping as a design direction, but do not establish the accuracy of the new action forecasts.

Figure 4 summarizes the reported read-path tradeoff. The accounting excludes fact acquisition, including properties extracted from an old session in response to a new query. This is not necessarily a one-time session cost. Full $C(q)$ in Eq. (7) must charge such work where it occurs and distinguish cold acquisition from warm reuse; the reported savings cannot yet be extended to that full ledger.

Finding 4: The budget-only prototype uses 368 fewer calls than Uniform-K=3; stopping saves another 88 at unchanged aggregate accuracy. These configurations motivate selective execution, while full acquisition costs remain outside the reported metric.

6.6 Exp-5. Which Mechanisms Matter

Table 8 groups prototype modules, not the three memory levels $\mathcal{H}, \mathcal{F}, \mathcal{U}$. The differences expose distinct roles for evidence organization, experience-guided access, and execution controls.

Organized facts and pipelines both matter. Removing the fact representation lowers accuracy by 6.1 points on MemBench and 2.7

Table 8: Prototype module ablations. Evidence organization, workload experience, and execution controls can affect accuracy and calls. These modules are not the three memory levels.

Configuration	MemBench		LME-v2	
	Score	Calls	Score	Calls
MemCatalog	**92.8**	**444**	**10.3**	**602**
Evidence framework				
w/o planner	89.1	900	8.8	903
w/o operators	87.9	900	8.3	903
w/o fact organization	86.7	900	7.6	903
Workload experience				
w/o workload experience	92.1	444	8.6	602
w/o priority ranking	92.5	444	9.3	602
w/o session eviction	92.8	444	10.1	651
Measured execution controls				
w/o pattern-aware budget	92.5	756	10.3	903
w/o early stopping	92.8	532	10.3	723

on LME-v2. Removing operators lowers it by 4.9 and 2.0; removing the planner lowers it by 3.7 and 1.5. Each configuration also issues more calls than the full prototype. The loss therefore cannot be explained simply by having fewer inference calls. Reusable identities and values provide the input to explicit computation; routing selects which computation to apply. The separately observed losses support both representation and processing. They are not additive contributions: changing one module changes the evidence and work seen by other modules. Nor does this table directly test recovery of a newly requested property from raw sessions.

Experience changes evidence use at the same call count. Removing workload experience loses 0.7 points on MemBench and 1.7 on LME-v2, while total calls remain 444 and 602. Removing priority ranking loses 0.3 and 1.0, again without changing those totals. These contrasts are especially relevant to our core claim: better memory operation need not mean more model calls. The larger score change on the open-answer benchmark is consistent with evidence choice affecting which facts reach the answer. It is not an error-type analysis, and the metric scales differ.

In the formal model, Eq. (4) makes the analogous access mechanism explicit: current relevance identifies eligible facts, while remembered use changes their order. The measured experience controls support workload-guided access, but do not separately verify the fact-version credit rule or rule out changes in cache contents.

Allocation and stopping have different effects. Without the measured pattern-aware budget, calls increase by 312 on MemBench and 301 on LME-v2; accuracy falls by 0.3 on the former and is unchanged on the latter. Without early stopping, calls increase by 88 and 121, with unchanged aggregate accuracy. Thus allocation can affect both quality and expense, whereas stopping is accuracy-neutral in these reported aggregates. The session-eviction control adds 49 calls on LME-v2 for a 0.2-point change. Its original scope is not documented here; it should not be interpreted as removing the durable raw evidence needed by the architecture in Section 3.

Table 9: Answer scores by benchmark-question position in a user's sequence, evaluated on users with ≥ 10 questions so that all four buckets are populated. MemCatalog increases monotonically across buckets; query difficulty and cache growth are not isolated. The subset differs from the all-user population in Table 3.

Method	Question position for one user			
	1–3	4–6	7–9	10+
EvoMem				
Full-Context	16.7	17.0	16.8	16.5
Mem0 [2]	20.1	20.5	20.4	20.2
Zep [11]	21.8	22.2	22.1	22.0
MemCatalog	**22.2**	**24.4**	**26.1**	**27.8**
DynamicMem				
Full-Context	28.7	28.8	28.6	28.4
Mem0 [2]	29.4	29.8	29.6	29.3
Zep [11]	**29.9**	30.0	29.9	29.8
MemCatalog	29.1	**30.3**	**31.7**	**32.8**

Finding 5: Representation and pipelines lose accuracy despite more calls when removed; workload experience loses accuracy at unchanged call totals. The two mechanisms cannot be reduced to a larger inference budget.

6.7 Exp-6. Does Access Improve Across a User's Queries?

Table 9 restricts to users with at least ten queries and groups each query by its position in that user's benchmark-question sequence. This subset is different from the full evaluation population; its absolute scores should not be substituted for the overall scores.

On EvoMem, MemCatalog rises from 22.2 to 27.8; on DynamicMem, from 29.1 to 32.8. The gains are 5.6 and 3.7 points. The strongest baseline, Zep, changes by only 0.2 and −0.1 points between the same endpoint buckets. Consequently, MemCatalog's gap to Zep widens from 0.4 to 5.8 on EvoMem and moves from −0.8 to 3.0 on DynamicMem. All four MemCatalog buckets increase monotonically, whereas the evaluated baselines show only small fluctuations.

This is the cross-question behavior second-order memory seeks to produce. Closed conversations extend raw history; queries add facts and operational experience. Alice's durations enrich content, while their reported use and acquisition cost guide later access. Content growth and operational experience are therefore distinct explanations of improvement.

The fixed-call experience ablations in Exp-5 and the position trend address complementary questions: experience affects scores without more aggregate calls, and scores improve as more experience becomes available. Together they support the role of persistent access feedback more strongly than either table alone. They still do not isolate fact-level use from cache growth and query difficulty. The backbone and prompt are fixed across buckets, but those other

Table 10: Prototype pattern-routing and control sensitivity on MemBench. The original agreement threshold θ and priority weight α use defaults θ=0.6 and α=0.5. Neither these controls nor the pattern labels evaluate the normalized plan partition or action-value policy.

Planner	Pattern acc.	Planner calls	End-to-end acc.
Rule-based (ours)	91.4	0	92.8
LLM planner	95.1	1	93.2
Always POINT	61.3	0	87.5
Parameter perturbation			
$\theta \in [0.4, 0.8]$	end-to-end acc. 92.3–92.9		
$\alpha \in [0.3, 0.7]$	end-to-end acc. 92.2–92.8		

factors are not. Shuffled-order, fixed-cache, and difficulty-matched controls are needed for a causal attribution to operational evolution.

Finding 6: The advantage widens across the observed position buckets. Together with fixed-call experience ablations, this supports persistent access adaptation, while leaving cache growth and query-order effects to be separated.

6.8 Exp-7. The Value and Cost of Routing

The rule-based router in Table 10 reaches 91.4% pattern accuracy at zero LLM calls. The LLM router reaches 95.1%, a 3.7-point increase, but improves end-to-end accuracy by only 0.4 points. Its extra call for each of 300 queries adds 300 calls, or 67.6% of the full prototype's 444-call count.

Routing accuracy and answer accuracy therefore do not improve one-for-one. A corrected route can still encounter missing source evidence or a wrong predicate judgment; some route differences may also leave the final answer unchanged. These are possible mechanisms, not observed error counts. By contrast, always choosing POINT loses 5.3 end-to-end points. The comparison supports making the required computation explicit, while showing a small measured return from further router complexity in this workload.

The lower block perturbs the original agreement threshold θ and priority weight α; accuracy ranges span at most 0.6 points. They control retained repetitions and evidence priority. The modest variation suggests tolerance, not statistical equivalence or freedom from calibration. Their definitions differ from normalized-plan routing and action-value allocation, so these scores do not validate those policies.

Finding 7: Selecting a nontrivial computation matters more here than paying for the more accurate router: the latter adds 300 calls for a 0.4-point end-to-end gain.

6.9 Exp-8. How History Length Changes the Tradeoff

Table 11 increases history from 40K to 1.5M tokens, a 37.5× range. Reported calls per benchmark question remain 1.9 for MemCatalog and 1.0 for Full-Context. Yet latency grows 20.7× for Full-Context, from 2.3s to 47.6s, and only 1.09× for MemCatalog, from 3.2s to 3.5s. Call count alone is therefore not a complete description of inference expense.

Table 11: Behavior as history grows on the three LongMemEval variants. The reported read-path calls and latency of MemCatalog are nearly flat in this configuration. This does not establish acquisition-inclusive, history-independent cost.

History	Calls / question	Latency (s)	Score
Full-Context			
~40K (LME-o)	1.0	2.3	49.3
~115K (LME-s)	1.0	5.8	43.6
~1.5M (LME-m)	1.0	47.6	58.5
MemCatalog			
~40K (LME-o)	1.9	3.2	67.2
~115K (LME-s)	1.9	3.4	64.9
~1.5M (LME-m)	1.9	3.5	75.2

At 40K, structured memory access is slower by 0.9s. At 115K, MemCatalog takes 3.4s against 5.8s, and at 1.5M it is 13.6× faster. This crossover makes the architectural tradeoff concrete: organizing and accessing evidence adds overhead, but need not repeatedly place the entire source history inside the answering prompt. Reusable facts and bounded evidence-processing steps provide a plausible explanation for the measured latency trend. The table does not report per-call token lengths or separate acquisition time, so it cannot directly attribute the trend to either one.

Accuracy advantages over Full-Context remain 17.9, 21.3, and 16.7 points across the three variants. The formal pipelines still require population coverage and binding evidence. The favorable measured tradeoff does not establish history-independent cost: new properties can require acquisition, and population work can grow with scope even under an execution cap.

Finding 8: The measured tradeoff shifts in favor of structured memory as history grows: call counts stay fixed while latency diverges. Coverage and acquisition obligations still limit any claim of constant full cost.

7 Related Work

Conversation and fact memory. MemGPT manages information across context and external memory tiers [10]. Mem0 builds persistent memory by extracting and updating salient conversational facts [2]. Zep represents time-dependent relationships through a temporal knowledge graph [11], while A-MEM builds linked notes that evolve with new experiences [18]. These systems develop fact and context representations. MemCatalog connects raw User–Agent history and query-induced facts with persistent Agent–Memory experience. This second-order state records exposure, reported use, and execution outcomes/cost, rather than another semantic description of the conversation.

Retrieval control and memory queries. LongMemEval separates memory design into indexing, retrieval, and reading, and evaluates information extraction, multi-session reasoning, temporal reasoning, knowledge updates, and abstention [16]. MemR^3 uses

an evidence-gap tracker and a controller that selects retrieval, reflection, or answering [3]. These precedents expose memory access and iterative evidence gathering. Our normalized-plan partition specifies the required computation; second-order experience changes evidence priorities, reuse, and budgets across the user's queries.

Query-driven acquisition. EUR [13] argues that incoming queries can reveal missing concepts and facts and leave reusable state for future workloads. We adopt query-driven acquisition for conversational evidence: first-order state retains the acquired facts, and second-order state retains their use and execution experience. Incoming information needs therefore change both content and later memory operations.

Repeated reasoning and computational budgets. Self-consistency samples multiple reasoning paths and aggregates their answers [14]. Agreement is useful evidence about variation among outputs, but cannot verify a shared false premise. Our execution policy conditions additional work on source support, evidence coverage, unresolved dependencies, and cost experience. It can reuse verified values without resampling, or spend on an unresolved predicate or source region. Action-level forecasts require comparison with fixed and adaptive sampling policies.

Evaluation of long-term agent memory. LoCoMo studies long-term conversational recall and related tasks across many sessions [8]. LongMemEval emphasizes multiple forms of memory reasoning [16], and MemBench broadens memory evaluation across scenarios and question types [12]. Their category labels are not normalized-plan classes. We distinguish benchmark-level observations from validation of dependency routing, fact-version credit, and action forecasts.

8 Conclusion

MemCatalog evolves both memory content and operations. Closed User–Agent conversations extend raw sessions; queries acquire first-order facts. Agent–Memory executions and validated use reports extend second-order experience, including empty-result outcomes and cost. Facts help the agent understand the user; experience helps memory understand how to serve the user. Single-fact retrieval, independent composition, and dependent composition expose the required computation. Cost-aware execution allocates work from current gaps and remembered outcomes, preserving partial or unknown support whenever an obligation remains.

The prototype evaluation covers eight benchmarks and eight LLM backbones, with gains up to 5.6 points on LME-s and 50.7% fewer read-path calls than Uniform-K=3 on MemBench at higher accuracy. These measurements support evidence processing, workload-guided access, and selective execution. Targeted validation of normalized-plan routing, fact-version attribution, action forecasts, and acquisition-inclusive costs remains necessary, as does reconciliation of the per-pattern aggregation. The interface retains explicit incomplete results wherever source or computation obligations remain unresolved.

References

[1] Anthropic. 2024. The Claude 3.5 Sonnet Model Card. https://www.anthropic.com/news/claude-3-5-sonnet.

[2] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv preprint arXiv:2504.19413* (2025).

[3] Xingbo Du, Loka Li, Duzhen Zhang, and Le Song. 2025. MemR3: Memory Retrieval via Reflective Reasoning for LLM Agents. *arXiv preprint arXiv:2512.20237* (2025).

[4] Google DeepMind. 2024. Gemini 1.5: Unlocking Multimodal Understanding across Millions of Tokens of Context. https://arxiv.org/abs/2403.05530.

[5] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. RULER: What's the Real Context Size of Your Long-Context Language Models? *arXiv preprint arXiv:2404.06654* (2024).

[6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *NeurIPS*.

[7] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023. Lost in the Middle: How Language Models Use Long Contexts. *arXiv preprint arXiv:2307.03172* (2023).

[8] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating Very Long-Term Conversational Memory of LLM Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. arXiv:2402.17753.

[9] OpenAI. 2023. GPT-4 Technical Report. https://arxiv.org/abs/2303.08774.

[10] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as Operating Systems. *arXiv preprint arXiv:2310.08560* (2023).

[11] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. *arXiv preprint arXiv:2501.13956* (2025).

[12] Haoran Tan, Zeyu Zhang, Chen Ma, Xu Chen, Quanyu Dai, and Zhenhua Dong. 2025. MemBench: Towards More Comprehensive Evaluation on the Memory of LLM-based Agents. In *Findings of the Association for Computational Linguistics: ACL 2025*. arXiv:2506.21605. doi:10.18653/v1/2025.findings-acl.989

[13] Nan Tang and Haixun Wang. [n. d.]. Back to the Future: The Evolving Universal Relation Model. Undated manuscript supplied as a reference.

[14] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *ICLR*. arXiv:2203.11171.

[15] Tianxin Wei, Noveen Sachdeva, Benjamin Coleman, Zhankui He, Yuanchen Bei, Xuying Ning, Mengting Ai, Yunzhe Li, Jingrui He, Ed H. Chi, Chi Wang, Shuo Chen, Fernando Pereira, Wang-Cheng Kang, and Derek Zhiyuan Cheng. 2025. Evo-Memory: Benchmarking LLM Agent Test-time Learning with Self-Evolving Memory. *arXiv preprint arXiv:2511.20857* (2025).

[16] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2024. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. *arXiv preprint arXiv:2410.10813* (2024).

[17] Wenya Xie, Shengming Zhou, Zelin Li, Pouya Parsa, Shuang Zhou, Xinheng Ding, Chinmay Arvind, and Guanchu Wang. 2026. DynamicMem: A Long-Horizon Memory Benchmark in Real-World Settings. *arXiv preprint arXiv:2606.22877* (2026).

[18] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-MEM: Agentic Memory for LLM Agents. *arXiv preprint arXiv:2502.12110* (2025).